

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИС- СЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа

Выполнил:
Гордеев Артём
К3339

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

Лабораторная работа №2: Реализовать микросервисную архитектуру бэкенд-приложения на основе проекта, спроектированного в ДЗ4.

Домашнее задание №5: Подключить и настроить Apache Kafka; реализовать межсервисное взаимодействие посредством очередей сообщений.

Лабораторная работа №3: Реализовать тестирование API средствами Postman; написать тесты внутри Postman в виде комплексного сценария по основному процессу приложения.

Ход работы

Реализация микросервисной архитектуры (ЛР2)

На основе спроектированной в ДЗ4 архитектуры реализованы пять микросервисов и API Gateway. Каждый сервис представляет собой отдельный Go-модуль с собственной БД PostgreSQL.

Структура проекта

Таблица 1 – Состав сервисов и их назначение

Сервис	Порт	Назначение
api-gateway	8080	Единая точка входа, валидация JWT, маршрутизация запросов
auth-service	8081	Регистрация, аутентификация, JWT
profile-service	8082	Профили соискателей/работодателей, компании
resume-service	8083	CRUD резюме с навыками, образованием и опытом
job-service	8084	Вакансии, отклики, спра-
notification-service	-	Потребитель событий Kafka

Контейнеризация

Все сервисы упакованы в Docker-контейнеры с многоэтапной сборкой (multi-stage build). Оркестрация выполняется через docker-compose.yml, который поднимает 11 контейнеров: 4 БД PostgreSQL, Zookeeper, Kafka, 5 микросервисов и API Gateway.

Листинг 1 – Пример Dockerfile (auth-service)

```
FROM golang:1.26-alpine AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN go build -o main .

FROM alpine:latest
WORKDIR /root/
COPY --from=builder /app/main .
```

```
CMD [ "./main" ]
```

API Gateway

API Gateway реализован на Go с использованием Gin и `httputil.ReverseProxy`. Валидация JWT выполняется один раз на уровне шлюза, после чего заголовки *X-User-ID* и *X-User-Role* прокидываются в целевой микросервис.

Листинг 2 – Маршрутизация в API Gateway (фрагмент)

```
v1 := r.Group("/v1")
v1.Any("/auth/*filepath", proxyHandler(authProxy))
v1.Any("/profile/*filepath", proxyHandler(profileProxy))
v1.Any("/companies", proxyHandler(profileProxy))
v1.Any("/companies/*filepath", proxyHandler(profileProxy))
v1.Any("/resumes", proxyHandler(resumeProxy))
v1.Any("/resumes/*filepath", proxyHandler(resumeProxy))
v1.Any("/jobs", proxyHandler(jobProxy))
v1.Any("/jobs/*filepath", proxyHandler(jobProxy))
v1.Any("/applications/*filepath", proxyHandler(jobProxy))
v1.Any("/industries", proxyHandler(jobProxy))
```

Межсервисное взаимодействие (синхронное)

Сервисы общаются по REST через внутреннюю Docker-сеть. Внутренние эндпоинты (`/internal/`) закрыты от внешнего доступа на уровне API Gateway.

Таблица 2 – Синхронные межсервисные вызовы

Потребитель	Поставщик	Эндпоинт	Цель
Job Service	Profile Service	GET <code>/internal/employers/:user_id</code>	Проверка компании работодателя
Job Service	Resume Service	GET <code>/internal/resumes/by-</code>	Проверка владения резюме
Profile Service	Auth Service	GET <code>/internal/users/:user_id</code>	Валидация пользователя

Интеграция Apache Kafka (Д35)

Инфраструктура

В `docker-compose.yml` добавлены контейнеры Zookeeper и Kafka (образы *wurstmeister*). При старте Kafka автоматически создаёт два топика: *user-events* и *job-events*.

Листинг 3 – Конфигурация Kafka в `docker-compose.yml`

```
kafka:
  image: wurstmeister/kafka:latest
  environment:
    KAFKA_ADVERTISED_HOST_NAME: kafka
    KAFKA_CREATE_TOPICS: "user-events:1:1,job-events:1:1"
    KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
  depends_on:
    - zookeeper
```

Продюсеры

Публикация событий реализована в двух сервисах с использованием библиотеки *segmentio/kafka-go*:

- **auth-service** – публикует событие *UserRegistered* в топик *user-events* при регистрации пользователя;
- **job-service** – публикует события *ApplicationSubmitted* и *ApplicationStatusChanged* в топик *job-events* при подаче отклика и изменении его статуса.

Публикация выполняется асинхронно через горутины (`go kafka.Publish...`), чтобы не блокировать HTTP-ответ клиенту.

Листинг 4 – Публикация события при подаче отклика (фрагмент)

```
go kafka.PublishApplicationSubmitted(
  jobID.String(),
  req.ResumeID.String(),
  job.EmployerID.String(),
)
```

Потребитель (notification-service)

Notification Service – отдельный микросервис, который подключается к Kafka как консьюмер (consumer group *notification-service-group*) и слушает оба топика. При получении сообщения десериализует JSON и обрабатывает событие в зависимости от поля *event_type*.

Таблица 3 – Обрабатываемые события

Событие	Топик	Действие
UserRegistered	user-events	Отправка приветственного
ApplicationSubmitted	job-events	Уведомление работодателя о новом отклике
ApplicationStatusChanged	job-events	Уведомление соискателя об изменении статуса

Листинг 5 – Обработка событий в notification-service (фрагмент)

```
switch eventType {
case "UserRegistered":
    email, _ := payload["email"].(string)
    role, _ := payload["role"].(string)
    fmt.Printf("[NOTIFICATION] Приветственное письмо для %s\n", email, role)
case "ApplicationSubmitted":
    fmt.Printf("[NOTIFICATION] Работодатель %s: Новый отклик на вакансию %s\n", employerID, jobID)
case "ApplicationStatusChanged":
    fmt.Printf("[NOTIFICATION] Соискатель %s: Статус отклика %s изменен на %s\n", jobSeekerID, appID, status)
}
```

Результат запуска

```
> docker-compose logs notification-service
notification-service-1 | 2026/05/25 20:39:18 Сервис уведомлений запускается... Брокер: kafka:9092
notification-service-1 | 2026/05/25 20:39:18 Начато прослушивание топика: job-events
notification-service-1 | 2026/05/25 20:39:18 Начато прослушивание топика: user-events
notification-service-1 | [NOTIFICATION] Отправка приветственного письма для employer_3qumee@test.com (Поль: employer)
notification-service-1 | [NOTIFICATION] Отправка приветственного письма для emp_d3m3cbatetest.com (Поль: employer)
notification-service-1 | [NOTIFICATION] Отправка приветственного письма для seeker_99u15n@test.com (Поль: job_seeker)
notification-service-1 | [NOTIFICATION] Отправка приветственного письма для emp_17luga@test.com (Поль: employer)
notification-service-1 | [NOTIFICATION] Отправка приветственного письма для seeker_moa9uc@test.com (Поль: job_seeker)
notification-service-1 | [NOTIFICATION] Работодатель 083a47dd-8876-4248-b5c1-afc7ef0a4f2ff: Новый отклик на вакансии 42462646-3698-4a61-9bbc-5d3d57c4ac33 с резюме ab881bb4-96f4-45ef-afc1-58ffa4b3ce16
notification-service-1 | [NOTIFICATION] Соискатель f23d9899-48b3-42c6-b1fe-4d02ecbe3850: Статус вашего отклика 1145ef20-e6af-424f-b2db-c484149f2aa7 изменен на viewed
notification-service-1 | [NOTIFICATION] Отправка приветственного письма для emp_lydqa@test.com (Поль: employer)
notification-service-1 | [NOTIFICATION] Отправка приветственного письма для seeker_uez6g@test.com (Поль: job_seeker)
notification-service-1 | [NOTIFICATION] Работодатель 64273115-868a-4825-8bd6-3bc25d50a9f1: Новый отклик на вакансии 120b43c7-7d06-4973-9c77-624bc0595bd5 с резюме 0d13c3ae-bbd4-4976-9c85-4ae87bc36427
notification-service-1 | [NOTIFICATION] Соискатель 269cb97e-a179-4279-9434-6db50d8602ef: Статус вашего отклика 48a857d4-a932-4a03-80ff-a23de50cae08 изменен на viewed
notification-service-1 | [NOTIFICATION] Отправка приветственного письма для seeker_f0aa7cc-00e5-4b13-930b-7715a1eb0f9ca@lab.ru (Поль: job_seeker)
notification-service-1 | [NOTIFICATION] Отправка приветственного письма для employer_aaa50f99-5ef3-4cc1-b391-a068bd4eb35a@lab.ru (Поль: employer)
notification-service-1 | [NOTIFICATION] Работодатель 83ac3567-e494-48e4-89e2-75c707b8cd01: Новый отклик на вакансии 5aa5ad7e-6434-4956-8164-caf275b2d3eb с резюме 2d28cd28-fb45-4843-9a6c-9c0f36843a0d
notification-service-1 | [NOTIFICATION] Соискатель c81376ef-2e57-4784-8fbb-f5e28de4ad6b: Статус вашего отклика 61725073-3811-44c8-8c24-9879ae3afe24 изменен на interview
notification-service-1 | [NOTIFICATION] Отправка приветственного письма для seeker_8cce8449-5038-4b58-9226-7d51bfaac912@lab.ru (Поль: job_seeker)
notification-service-1 | [NOTIFICATION] Отправка приветственного письма для employer_7c3be70d-ef16-4ebb-8d60-f6d4ccf8b602@lab.ru (Поль: employer)
notification-service-1 | [NOTIFICATION] Работодатель a9a371d5-6a9e-4873-9ac3-1c26a30f93ab: Новый отклик на вакансии 31e6ddb8-f475-4edf-8105-7c51c6ee3e67 с резюме a3dc788b-90c1-4122-b83a-b73fb164d15f
notification-service-1 | [NOTIFICATION] Соискатель 1171acf2-b0d1-4732-93b5-07c0a8602d67: Статус вашего отклика 59971d96-61dd-4da8-99eb-7d90851e69c3 изменен на interview
notification-service-1 | [NOTIFICATION] Отправка приветственного письма для emp_8xnrg@test.com (Поль: employer)
notification-service-1 | [NOTIFICATION] Отправка приветственного письма для seeker_n31n5b@test.com (Поль: job_seeker)
notification-service-1 | [NOTIFICATION] Работодатель 41af1a68-692b-46b6-b5b3-3b96d14fcc1f: Новый отклик на вакансии 50a78408-e834-42c4-8a3c-a0033dd66999 с резюме fd2a6a91-2e6a-4e9e-8576-39364faab61d
notification-service-1 | [NOTIFICATION] Соискатель 8bca1869-917b-4ed6-a8d6-a28a4c91debb: Статус вашего отклика 1d97d0a8-2995-4e01-8787-0348b44b5910 изменен на viewed
```

Рисунок 1 – Логи notification-service с обработанными событиями

Тестирование API средствами Postman (ЛР3)

Комплексный сценарий

Создана коллекция реализующая сквозной E2E-сценарий основного бизнес-процесса. Коллекция включает 29 запросов, покрывающих все эндпоинты системы.

Таблица 4 – E2E-сценарий тестирования

Шаг	Метод	Эндпоинт	Роль
1	POST	/auth/register	Работодатель
2	POST	/auth/login	Работодатель
3	POST	/profile/employer	Работодатель
4	GET	/profile/employer	Работодатель
5	PUT	/profile/employer	Работодатель
6	POST	/companies	Работодатель
7	GET	/companies/:id	-
8	PUT	/companies/:id	Работодатель
9	GET	/industries	-
10	POST	/jobs	Работодатель
11	GET	/jobs	-
12	GET	/jobs/:id	-
13	PUT	/jobs/:id	Работодатель
14	POST	/auth/register	Соискатель
15	POST	/auth/login	Соискатель
16	POST	/profile/job-seeker	Соискатель
17	GET	/profile/job-seeker	Соискатель
18	PUT	/profile/job-seeker	Соискатель
19	POST	/resumes	Соискатель
20	GET	/resumes	Соискатель
21	GET	/resumes/:id	-
22	PUT	/resumes/:id	Соискатель
23	POST	/jobs/:id/apply	Соискатель
24	GET	/applications/me	Соискатель
25	GET	/jobs/:id/applications	Работодатель
26	PATCH	/applications/:id/status	Работодатель
27	DELETE	/applications/:id	Соискатель
28	DELETE	/resumes/:id	Соискатель
29	DELETE	/jobs/:id	Работодатель

Обеспечение идемпотентности

Для повторяемости прогонов при регистрации используется генерация случайного email через Pre-request Script:

Листинг 6 – Pre-request Script для генерации уникального email

```
let randomString = Math.random().toString(36).substring(7);
pm.collectionVariables.set('employer_email', 'emp_' + random-
String + '@test.com');
```

Тестовые скрипты

В каждом запросе настроены тесты во вкладке Tests. Проверяются HTTP-статусы и наличие обязательных полей. Идентификаторы и токены извлекаются и сохраняются в переменные коллекции для передачи в последующие запросы.

Листинг 7 – Пример тестового скрипта (Login)

```
pm.test('Status code is 200', function () {
    pm.response.to.have.status(200);
});
let jsonData = pm.response.json();
pm.collectionVariables.set('employer_token', jsonData.access_to-
ken);
```

Результат прогона

БР ла62 - Run results

Ran today at 07:11:20 PM · [View all runs](#)

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	none	1	992ms	29	0	17 ms

RUN SUMMARY

	1
> POST 1. Register Employer	1 0
> POST 2. Login Employer	1 0
> POST 3. Create Employer Profile	1 0
> GET 4. Get Employer Profile	1 0
> PUT 5. Update Employer Profile	1 0
> POST 6. Create Company	1 0
> GET 7. Get Companies	1 0
> PUT 8. Update Company	1 0
> GET 9. Get Industries	1 0
> POST 10. Create Job	1 0
> GET 11. Get Jobs	1 0
> GET 12. Get Job by ID	1 0
> PUT 13. Update Job	1 0
> POST 14. Register Job Seeker	1 0
> POST 15. Login Job Seeker	1 0
> POST 16. Create Job Seeker Profile	1 0
> GET 17. Get Job Seeker Profile	1 0
> PUT 18. Update Job Seeker Profile	1 0
> POST 19. Create Resume	1 0
> GET 20. Get My Resumes	1 0
> GET 21. Get Resume by ID	1 0
> PUT 22. Update Resume	1 0
> POST 23. Apply for Job	1 0
> GET 24. View Applications (Job Seeker)	1 0
> GET 25. View Applications (Employer)	1 0
> PATCH 26. Update Application Status	1 0
> DELETE 27. Delete Application	1 0
> DELETE 28. Delete Resume	1 0
> DELETE 29. Delete Job	1 0

Рисунок 2 – Результат запуска коллекции в Postman Collection Runner

Вывод

В ходе выполнения лабораторных работ и домашнего задания реализована микросервисная архитектура приложения UnhappyJob, спроектированная в Д34. Система включает пять Go-микросервисов с отдельными базами данных PostgreSQL, объединённых через API Gateway с JWT-аутентификацией.

Для асинхронного межсервисного взаимодействия интегрирован Apache Kafka с двумя топиками. Реализован сервис уведомлений, потребляющий события регистрации пользователей, подачи откликов и изменения их статусов.

Тестирование API выполнено через Postman с коллекцией из 29 запросов, покрывающих все эндпоинты системы. Сценарий идиempotentен и проходит без ошибок при повторных запусках.